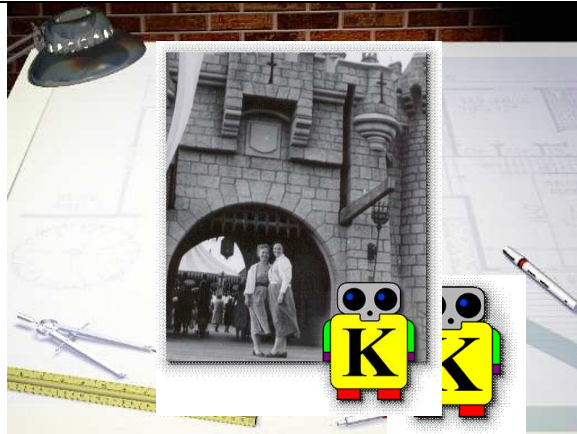




Slide 1

CS 170

Java Programming 1

Extending Karel

Duration: 00:00:50

Advance mode: Auto

Notes:

Hello, welcome to the CS 170, Java Programming 1 lecture, Extending Karel. When we last saw Karel, he was busy working as a security guard, patrolling the castle at Disneyland. When originally worked on the Patrol the Castle Project you learned how to use functional decomposition to break a complex task into simpler methods, like `patrolWall()` and `patrolTurret()`.

In this lesson you'll be re-introduced to the topic of Inheritance and learn how to create more specialized robot classes. The new classes you'll create will make use of some predicate methods to perform more complex tasks. You'll also learn how to add constructors to customize your robots when they are created.

Karel Returns

- Open BlueJ and the ic12 project.
- **Exercise 1:** Create a `PatrolTheCastle` object on the workbench and call its `task` method. Snap a picture when Karel is finished working.
- So, how does Karel's boss know that he just didn't come to work and fall asleep?

Slide 2

Karel Returns

Duration: 00:01:18

Advance mode: Auto

Notes:

To get started with this lesson, open up BlueJ and the ic12 project we've been working on that contains the `PatrolTheCastle` and `CastleGuard` programs. Start a new section in your in-class exercise document and then, for Exercise 1, create a `PatrolTheCastle` object on the workbench. You can name it anything you like.

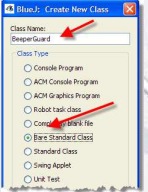
Call the object's `task()` method and make sure Karel goes completely around the castle. When he's finished and turned off, as shown in the picture here, snap me a screen-shot and paste it into your document.

Notice that I've called the `World.setDelay(25)` method to speed Karel up a bit for these larger programs. If you want to make him go faster, just lower the delay factor in `PatrolTheCastle`.

Now, suppose that you're Karel's boss and you come to work and find him sleeping like this right beside the drawbridge. How do you know that Karel actually completed his job? You don't, right? To give the boss some peace of mind, let's modify Karel so that he drops a beeper each time he moves.

The BeeperGuard

- We created a specialized version of **OCRobot** named **CastleGuard** using **inheritance**. Let's create a new version that drops a beeper each time it moves. That way the boss can check up on him.
- Exercise 2:** add a new Bare Standard Class to your project, and name it **BeeperGuard**. Then, open the file in your editor.



Slide 3
The BeeperGuard
 Duration: 00:01:00
 Advance mode: Auto

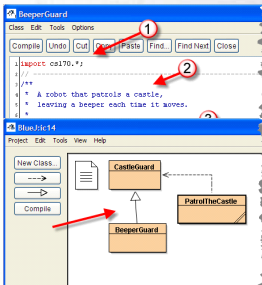
Notes:

The CastleGuard class is a specialized version of the OCRobot class that we developed by using inheritance. We then gave our new class additional capabilities by defining new methods, such as patrolWall(). Today, we're going to create another new class, again using inheritance. This time, we'll use the CastleGuard class as the base or superclass, instead of OCRobot, so we don't lose any of those new features we developed. Our new robot will drop a beeper every time he moves so that his boss can check up on him and make sure he didn't take any shortcuts in his rounds.

For Exercise 2, create a new class named BeeperGuard. Use the Bare Standard class template. When the class icon show up in the project window, double-click it to open it in your editor. The next slide has the changes you need to make for this exercise.

Setting Up BeeperGuard

- Add these changes to **BeeperGuard** and compile:
 - `import cs170.*;` on line 1
 - `extends CastleGuard` on the header line
 - Add a comment and your name at the top of the class
- Note that **BeeperGuard** is now a specialized kind of a **CastleGuard**



Slide 4
Setting Up BeeperGuard
 Duration: 00:01:23
 Advance mode: Auto

Notes:

Inside your editor, make these changes to BeeperGuard. When you're finished, compile the BeeperGuard class.

- At the very top of the file, on line 1, add `import cs170.*;`; This will import the OCRobot definitions we'll use.
- On the class header line, after the words `public class BeeperGuard`, add the phrase `extends CastleGuard` to indicate that our class will use CastleGuard as a starting point.
- Finally, add a comment to the top of the file explaining what the new robot class will do. Also, add your name and date.

Shoot me a screen-shot (for Exercise 2) showing these changes.

Note that the BeeperGuard class is now a specialized kind of CastleGuard: that is, anything a CastleGuard can do, like patrolling a turret, the BeeperGuard can also do; you don't have to teach it, it simply inherits those abilities. That also means that anywhere we previously employed a CastleGuard, we can substitute that instance with a BeeperGuard object.

Let's see how that works by creating a new BeeperGuard and putting it to work patrolling the castle.

Setting Up the Task

- Add these changes to **PatrolTheCastle**
 - Change **CastleGuard** variable to **BeeperGuard**
 - Initialize karel using the **BeeperGuard()** constructor

```

1  * @version Spring 2008
2  */
3  public class PatrolTheCastle {
4  {
5      // Instance/static variables
6      private BeeperGuard karel = new BeeperGuard();
7  }
8  }
9  }
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

- **Exercise 3:** create a **PatrolTheCastle**, call **task()**

Slide 5

Setting Up the Task

Duration: 00:01:01
Advance mode: Auto

Notes:

Open the PatrolTheCastle robot task program and make these two changes:

- First, change the type of the instance variable karel from a CastleGuard to a BeeperGuard on line 13, as shown here.
- Second, use a BeeperGuard constructor to initialize the variable on line 26, as shown here.

Notice that nothing else needs to change at all. Since the BeeperGuard class inherits all of the capabilities of the CastleGuard class, you can send exactly the same messages to the new robot and it will respond to them.

Just to silence any lagging doubts about that though, create a PatrolTheCastle object once again and call its task() method to make sure it still works. When you're done, shoot me a screen-shot of your code with the two changes asked for.

Inheritance and Overriding

- Subclasses **inherit** all of their parent's public methods
 - Use methods just as if they were defined in subclass
 - Any thing a superclass can do, a subclass can do
- The subclass can also
 - Add entirely new methods
 - Replace or redefine existing methods
 - Add new behavior to existing methods
- The last two techniques employ method **overriding**
 - Redefine a parent method in the child class
 - Use exactly the same signature in both classes

Slide 6

Inheritance and Overriding

Duration: 00:02:32
Advance mode: Auto

Notes:

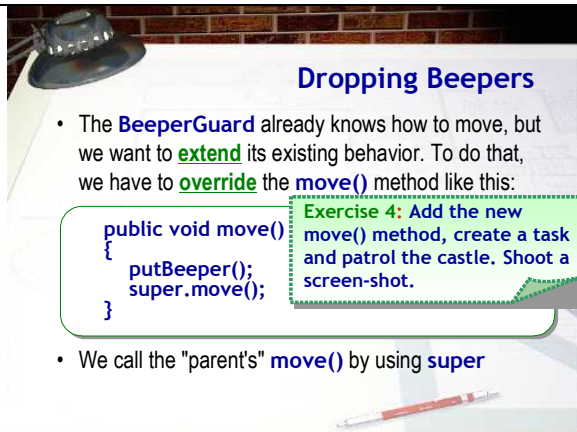
As you can see, when you create a new subclass (or child class), it automatically inherits all of its parent's public methods. Even though the BeeperGuard class doesn't contain a definition for the turnRight() method, you can still use it because the parent class, CastleGuard contains such a definition, and the BeeperGuard class inherits it.

We call this an IS-A relationship. Anything a CastleGuard can do, a BeeperGuard can do, because every BeeperGuard IS-A CastleGuard. This is also called the principle of substitutability.

However, you might have noticed that even though our BeeperGuard can do everything that the CastleGuard class can do, as yet, it can't do any more. When we use inheritance, and the extends keyword, we really want to add functionality in some way; we want to make better robots, not just robots with a different class name. To do that, our new sub-class can:

- First, you can add entirely new methods to create specialized versions of existing classes. If you create an ImageButton class from the Button class, you'd probably give it a method named drawImage(), something the parent Button class would have no use for.
- Second, you can change the way that the new class responds to the same messages by redefining or replacing a public method that the parent class already contains. In other words, the new class would respond to exactly the same message as the parent class, it would just do things in a different way.
- Finally, you might want to add something to, or extend the way that one of the parents methods works. This is kind of a combination of plain inheritance (since the parent's behavior continues on unchanged) and redefinition, since you add new behavior to the parent's behavior.

These last two techniques are carried out by employ a feature known as method overriding, not to be confused with method overloading. With method overriding, you create a new method in a subclass that has exactly the same signature (method name and parameter list) as a method in the super or parent class. (With method overloading, remember, you defined a second method of the same name that had a different signature or parameter list.)



Dropping Beepers

- The **BeeperGuard** already knows how to move, but we want to **extend** its existing behavior. To do that, we have to **override** the **move()** method like this:

```
public void move()
{
    putBeeper();
    super.move();
}
```

Exercise 4: Add the new **move()** method, create a task and patrol the castle. Shoot a screen-shot.

- We call the "parent's" **move()** by using **super**

Slide 7

Dropping Beepers

Duration: 00:02:07

Advance mode: Auto

Notes:

So, let's put this feature to work in our **BeeperGuard** class. **BeeperGuard** already has a method named **move()** that it inherits from **CastleGuard**. This is the method used by **PatrolTheCastle** to send Karel about his rounds. To teach Karel how to drop beepers:

- we have to keep the **move()** method because that's the method that **PatrolTheCastle** calls.
- we don't want to completely replace it because we still want Karel to move just the way he always has.
- we do want to extend it by adding additional behavior to the **move** method.

Here's how we do that:

- Define a method in the **BeeperGuard** class called **move()** that looks like this. This is called overriding the method, since the parent (actually great-great-great grand-parent) robot class defined exactly a method with this behavior.
- Add the new features you want. Here, I've called the **putBeeper()** method because I want to drop a beeper each time our robot moves.
- Finally, we need to invoke the **move** method inherited from our ancestors. Unfortunately, you can't just write **move()**. If you do that, then the **move()** method will call itself. This is called recursion, and it can be useful, but it's not what we want right now. Instead, you need to specify that you want to invoke the "parent" version of the **move()** method from inside the "child" version of **move()** by using the keyword **super** as the method receiver, like this.

Let's see if it works. For Exercise 4, create a **PatrolTheCastle** object and call its **task** method. You'll see that this creates a problem. Locate the problem in the Karel console window and shoot me a screen-shot. After you're done, we'll fix the problem.

What's Wrong?

- The **BeeperGuard** has no beepers in its bag
 - Give it some beepers by adding a constructor

```
public BeeperGuard(int st, int ave,
                  Direction facing, int beepers )
{
    super(st, ave, facing, beepers);
}
```

- Doesn't compile, since **CastleGuard** has no constructor either. Copy same constructor to **CastleGuard** (change name) and compile.

Slide 8

What's Wrong?

Duration: 00:02:21

Advance mode: Auto

Notes:

The reason our program doesn't work is because the **BeeperGuard** class doesn't have any beepers in its beeper-bag. To fix that, all we need to do is to add a constructor like this: this constructor allows us to place our robot on a particular street and avenue, specify the location it's facing (using the constants **NORTH**, **SOUTH**, etc. defined in the **Direction** class, which is part of the **cs170** package), and indicate how many beepers it should contain.

Once you've typed this in, you're immediately faced with another problem. What should you do inside the constructor? The fields for street, avenue and number of beepers aren't part of the **BeeperGuard** class definition. Instead, they're defined in a distant ancestor class called **UrRobot**. And, when the instance variables were defined, just like all the instance variables you've defined, they were made private, so you couldn't initialize them if you wanted to.

So, how do we initialize them? By using a constructor: in this case, the parent class constructor. Instead of using the parent class name, like you'd normally do with a constructor, you use the name **super**, just like you were calling a method. Here's the line to add.

Now, when you add this, all of a sudden the class no-longer compiles. By calling the **super()** constructor, you're telling Java to look in the parent class **CastleGuard**, for a constructor with this signature. The only problem? We didn't add a constructor to **CastleGuard**. To fix this, just copy the same constructor to the **CastleGuard** program and change the name. You don't have to change its contents at all. Then, go ahead and compile.

Notice the two uses we've made in this lesson of the word **super**. Write **super.move()** to call a superclass method from a subclass method. Write **super()** to calling a superclass constructor from a subclass constructor.

Starting Work

- Now Karel can "teleport" to work. No need for the **goToWork()** method. Comment out the method from **CastleGuard**, comment out the method call from **PatrolTheCastle**, and create karel like this:

```
karel = new BeeperGuard(4, 3, North, infinity);
```

- Exercise 5:** patrol the castle and shoot a picture.
- What happens if Karel goes around twice?

Slide 9

Starting Work

Duration: 00:01:15

Advance mode: Auto

Notes:

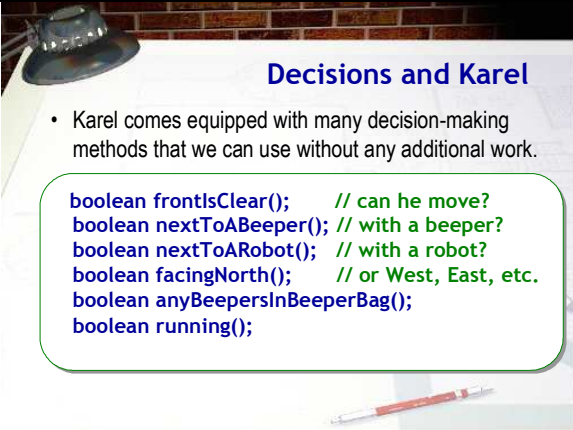
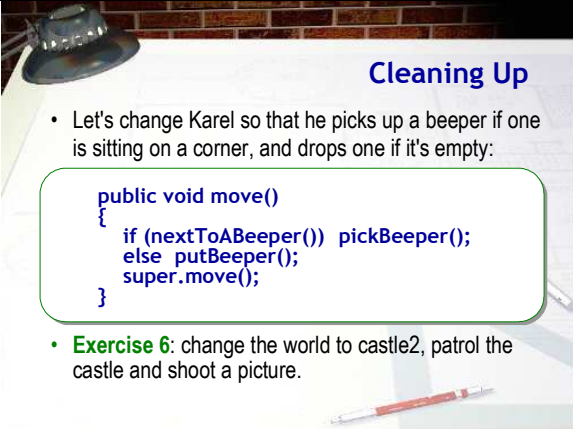
Now that we have constructors for our class, Karel no longer needs to start out at position 1,1 facing north. We can teleport him to work. That means we no longer need the **goToWork()** method at all.

- comment out the method from the **CastleGuard** class and
- comment out the method call from **PatrolTheCastle**.

Now, locate the place where karel is initialized and use your new constructor to create him ex-nihilo, ready to report for work and fully loaded with beepers.

For Exercise 5: patrol the castle and shoot a picture.

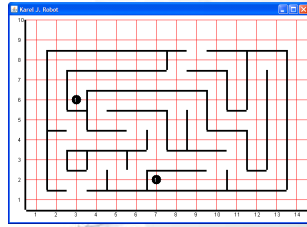
Now, try having karel patrol the castle twice. (You'll need to modify **PatrolTheCastle** a little). Shoot me a picture of the result. Notice that the beepers now have a little 2 on them,

	indicating that the beepers are piling up on the corner. That obviously can't be good. let's see if we can make Karel a little greener and get him to recycle his beepers. To do that, he'll have to learn to make some decisions.
 Slide 10 🎧 Decisions and Karel Duration: 00:00:51 Advance mode: Auto	Notes: Karel inherits several predicate methods, methods that return a boolean value. Karel can use those methods to make decisions about whether to move, which way to move, or whether to pick up or drop a beeper. Here are some of the methods: frontIsClear() is true if Karel can move forward. He can use nextToABeeper() and nextToARobot() to find out who's sharing his corner. Karel can find out which direction he's going with the facingNorth(), facingEast(), facingWest() and facingSouth() methods. The anyBeepersInBeeperBag() method returns true if he has, well, any beepers in his beeper bag. Finally, the running() method returns true if Karel is running. Of course, if it returns false, there's not much of anything he can do about it.
 Slide 11 🎧 Cleaning Up Duration: 00:00:52 Advance mode: Auto	Notes: Let's finish up this lesson by changing the BeeperGuard class so that Karel only picks up a beeper if there is one on the corner. If the corner is empty, he'll drop it. To do this, you only need to change the code you added to the move method, replacing it with an if-else statement. Notice that the parent's move method (super.move()) still remains the same. Once you've made this change, switch to PatrolTheCastle. Change the last part of the scenario URL to "castle2.kwld". Then create an instance of the task class, call the task method, and shoot a picture. Notice that Karel now picks up the beepers on the corners that contain them and drops them on the empty corners.



Express Yourself

- Create a robot to walk the fixed maze shown here:
 - **MazeWalker** extends **OCRobot**, add any methods
 - Make sure you enter at 1,3 and leave at 9,9
 - Walk entire maze, pick up beepers where you find them
 - Change URL to **"lab02h.php"** and run again to make sure you get all beepers.



Slide 12

Express Yourself

Duration: 00:01:08

Advance mode: Auto

Notes:

That's it for the lecture-lab exercises. For one of your homework assignments, though, I want you to complete a more complex problem. I want you to use complete the starter class, WalkMaze. You should define an OCRobot subclass, MazeWalker, to walk through this fixed maze. The maze will not change. You should always enter at 1st Street, 3rd avenue and leave at 9th Street 9th Avenue.

Somewhere in the maze are two beepers. Use the predicate methods to recognize them and pick them up. When you have walked through the entire maze, turn Karel off.

Once your have both classes complete, change the scenario URL to "lab02h.php" which will place the two beepers at random locations somewhere in the maze.

Check this week's exercise page for information on submitting and testing this assignment.